# Project
# Advanced Computerarchitecture

(B-KUL-JLI1QS)

Reinout Annaert, Michiel Van Nieuwenhove

May 9, 2024




Session Date: 09/05/2024

Class: MElsw

Instructor: Alex Marinšek

# 1 Introduction

This report marks the culmination of a lab series conducted as part of the Advanced Computer Architecture course. The focus of these lab sessions has been on exploring parallel computing on GPUs using CUDA. Specifically, this report documents the findings and outcomes of a project centered around raycasting, a fundamental technique in computer graphics. Throughout this project, we have delved into the intricate details of 2D raycasting, their optimization for parallel execution, and their implementation on CUDA-enabled architectures. By leveraging the power of parallel computing, we aimed to accelerate the raycasting process, thereby enhancing its performance and applicability in graphical applications.

# 2 Problem statement

In the context of the project *Ingenieursbeleving 2* within the second year of the engineering program, student groups embarked on a challenging task: developing a Python game incorporating a 2D raycaster. This game, designed to be controlled using a dramcontroller facilitated by I2C communication, required a blend of programming skills, creativity, and understanding of engineering principles.

The primary focus of this report centers on the software aspect of the project, specifically delving into the intricacies of the 2D raycaster component. Although it was enjoyable to develop the game, a significant challenge arose during development: the fluctuating frames per second (FPS). These fluctuations were notable, with FPS values ranging from a minimum of 1 to a maximum of 15. This erratic performance not only hindered the gaming experience but also raised questions about the efficiency and optimization of the underlying code.

Thus, the core objective of this report is to investigate how CUDA and parallel computing techniques can be harnessed to address and ultimately enhance the performance of the 2D raycaster. By leveraging parallel processing capabilities, we aim to optimize the computational workload and streamline the rendering process, thereby achieving smoother gameplay and a more consistent FPS rate.

Before diving into the intricacies of CUDA and parallelization, a crucial preliminary step involves translating the existing Python code into C++. Once the code translation is complete, our focus shifts towards parallelization using CUDA.




Figure 1: Python game Ingenieursbeleving 2 (2022)

# 3 2D raycasting

The fundamental concept behind 2D raycasting involves projecting a single ray through each vertical line in the camera's view instead of through every pixel. Scenes in this context are constructed from uniform primitives, such as cubes, and lack height disparities within the game world.

The inception of 2D raycasters traces back to their utilization in the iconic game Wolfenstein in 1992. The game environment consisted solely of cubes of uniform size, devoid of genuine 3D objects. Objects were rendered using sprites, which are 2D images placed and scaled within the scene based on their distance from the camera.




Figure 2: Wolfenstein (1992)

In the realm of 2D raycasting, the game world is represented through its own coordinate system, typically depicted by a 2D matrix. Each cell within this matrix symbolizes a 1x1x1 cube within the game world.

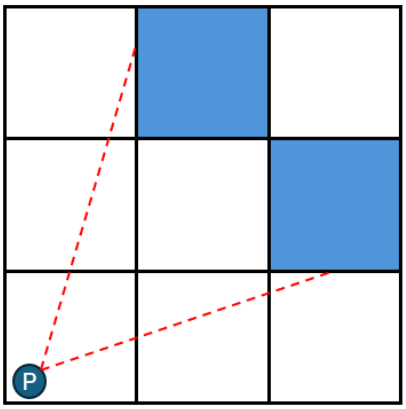



Figure 3: Player position in a 2D grid

The raycasting algorithm consists of three steps. The GitHub repository containing the source code can be found at *Github Repository*.

1. For each column on the screen, calculate the direction of the ray to be cast through that column of the screen. The camera plane is positioned at a distance `d_camera` from the player.

2. Next, we need to find the first wall that the ray intersects with and determine its distance from the player. To achieve this, we calculate all intersection points between the ray and the cells in the grid. While various algorithms exist for this purpose, in our project, we utilized the DDA algorithm. In this algorithm, the distance between consecutive intersections with horizontal lines remains constant, as does the distance between consecutive intersections with vertical lines.

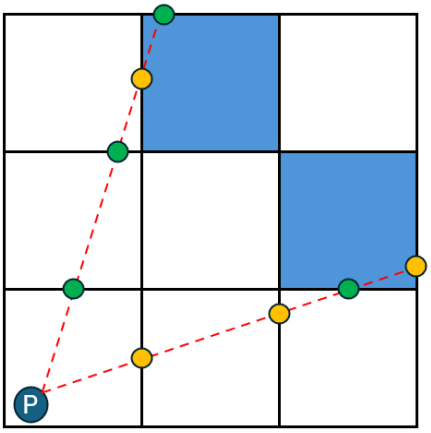



Figure 4: Intersection points

3. Now that we know which wall we've hit and its distance, we need to render that wall. The length of the wall segment to render is inversely proportional to the distance to the wall. However, 2D raycasters suffer from fisheye distortion because the algorithm calculates the Euclidean distance between the player and the wall. A wall that is parallel to the camera plane should appear equally tall in all screen columns. To address this issue, we use the perpendicular distance to the camera plane instead of the Euclidean distance to the player.

   To apply textures to walls, we must translate texture coordinates into screen coordinates. We have access to the texture coordinates for all vertices of each polygon. By interpolating these coordinates, we can determine the texture coordinates for all points within the polygon. This interpolation process is straightforward. Let's consider a texture that is W pixels wide and H pixels high. In this case, the left side of the wall corresponds to texture coordinate 0, and the right side corresponds to texture coordinate W. As for the Y-coordinate of the texture, it does not require explicit determination since we can directly apply the entire column X of the texture onto the wall column we are rendering. However, rescaling becomes necessary to ensure that the height of the texture column matches the height of the wall column, which is dictated by the distance to the wall.

# 4 Parallelization

The 2D raycaster, when executed by the `CPU`, follows a sequential process in its main loop comprising three steps. Initially, the screen is cleared, followed by computation of distances per column, and finally, the results are rendered onto the screen. Each of these steps involves iterating through all columns on the screen. The rationale for parallelization lies in employing one thread per pixel column, facilitating simultaneous calculation of all distances.

In the initial GPU implementation (`GPU1`), only the second and third steps of the aforementioned process are parallelized. This necessitates the transfer of essential parameters to the GPU, including constants like the screen and wall texture, which require copying only once, and variables such as player position and rotation, which need to be transferred every frame. Pixel values for each pixel are stored in the GPU's global memory and subsequently copied back to the CPU after thread completion.

An enhancement to this approach involves migrating the screen-clearing task to the GPU (`GPU2`). Initially, an attempt was made to leverage thread divergence, assigning the first thread the responsibility of clearing the screen. The initial reason to do this, was to not have to copy the contents of the screen every frame to GPU. However, this approach incurred inconsistencies due to concurrent memory access by multiple threads. Furthermore, the heavy workload assigned to the first thread became a bottleneck, significantly impeding code execution speed. These challenges were addressed by allocating each thread the task of clearing its respective column before proceeding to distance calculation. This divides the work between threads and keeps their memory access separated.

To ensure program functionality in case there are insufficient threads, striding is implemented. This is attempted in two manners. In the first approach, each thread is initialized to the pixel index it would normally possess. Instead of exclusively executing for this index, a loop is employed. The thread processes all columns of its own index plus k times the number of threads until it reaches the screen width. For the second implementation, threads process a predetermined number of sequential columns. Initially, each thread computes its index and the quantity of columns it needs to process. Figure 5 illustrates the disparity between these two implementations.

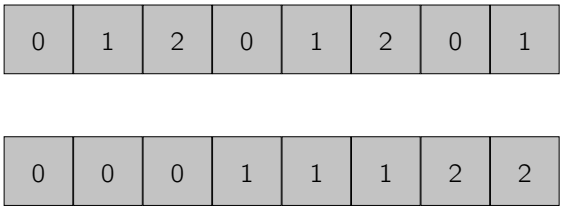



Figure 5: Two different methods of implementing striding. 3 threads are considered: 0, 1 and 2. The order of the number represents the order in which they calculate columns.

# 5 Experiments and evaluation

In this section, we conduct experiments with the raycaster implementations. Based on these experiments, we draw conclusions.

## 5.1 Performance of three implementations

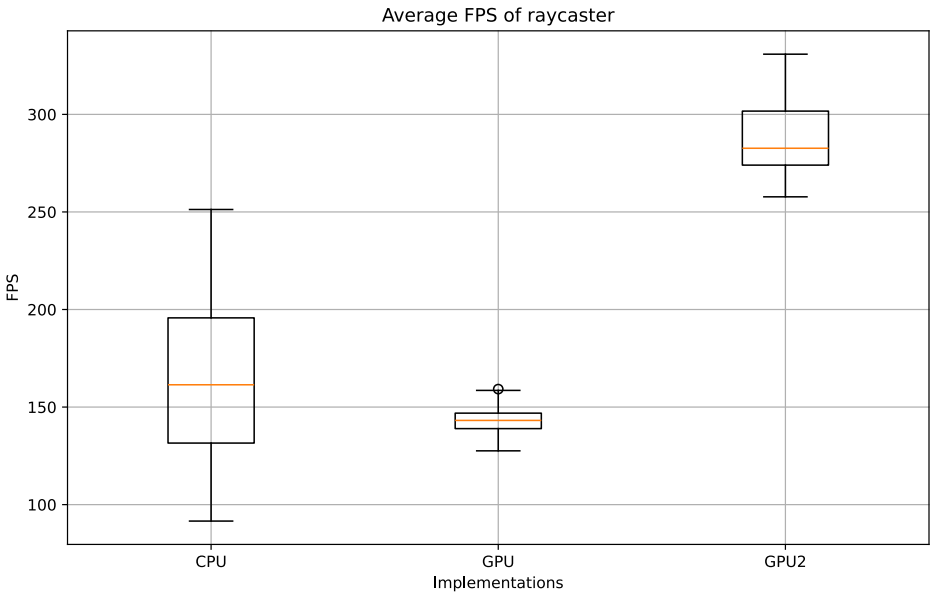



Figure 6: FPS of the three raycaster implementations. `GPU1` and `GPU2` use 3 blocks with a blocksize of 512

To compare the performance of the three raycaster implementations, we measure the FPS while moving within a predefined world where identical walls are rendered. We set the screen size to 1280x720. We present the measurements in a box plot in Figure 6 to illustrate the FPS.

From the experiments, it shows that the CPU implementation (`CPU`) has the largest fluctuations in FPS, with the lowest worst-case FPS among the implementations. The first GPU implementation (`GPU1`) exhibits minimal FPS fluctuations, though its median is slightly lower than that of the CPU implementation. On the other hand, the second GPU implementation (`GPU2`) achieves significantly higher FPS, with fluctuations slightly higher than the first GPU implementation but lower than those of the CPU implementation. These FPS measurements suggest that screen clearing in the first GPU implementation posed a bottleneck for FPS, resulting in substantially lower FPS, with the median even lower than that of the CPU. We can conclude that by parallelizing the calculations per column and parallelizing the screen clearing, significant FPS gains can be achieved compared to the CPU implementation.

## 5.2 Striding

The performance of the two different striding methods is illustrated in figure 7. Grouped striding requires one additional operation to execute, resulting in a slight performance difference, as depicted in the boxplot. Since both methods utilize coalesced memory, there is no advantage to be gained in that aspect.

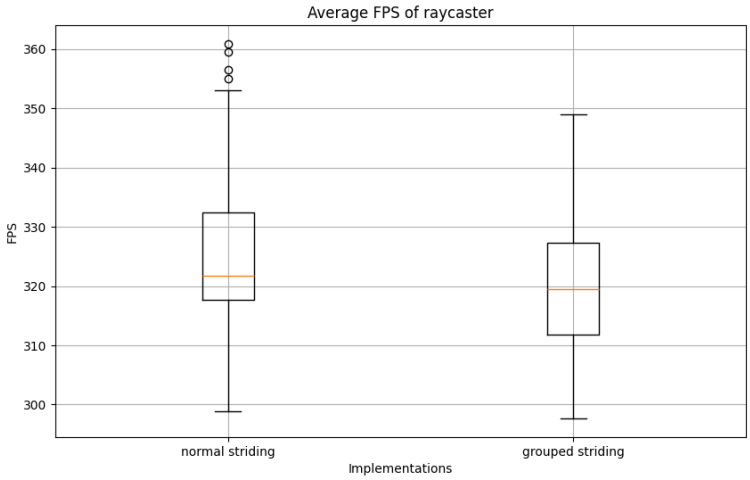



Figure 7: FPS as a function of the blocksize for the different striding methods (GPU2)

## 5.3 Influence blocksize

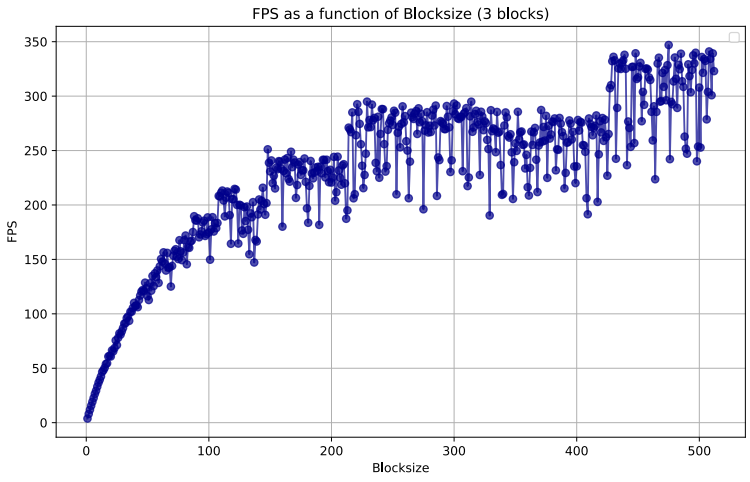



Figure 8: FPS as a function of the blocksize (GPU2)

To investigate the impact of blocksize on FPS while keeping the number of blocks constant, we conduct the following experiment. We focus the player's view towards several walls and fix this field of view throughout the experiment. Then, we increment the blocksize by 1 every 5 seconds, recording the average FPS over that 5-second interval. The results of this experiment are displayed in the graph in Figure 8.

In the graph, we observe minimal fluctuations until a blocksize of around 100. Up to this point, the limited number of threads presents a bottleneck. As the blocksize increases, we notice a corresponding increase in FPS, but also a higher level of fluctuation in FPS. Remarkably, there are steps in the graph, indicating sudden large increases in FPS at certain blocksize transitions. To verify the steps, we repeated the experiments, and each time, the steps were consistently observed at those specific block sizes.

To explain why this graph looks like this, it is easier with a smaller example. Suppose there is a screen with five columns. By incrementing the number of threads from 1 to 5, the threads executing the columns appear as follows:

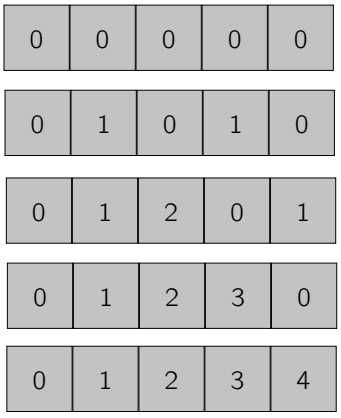



Figure 9: Threads per column for an example with 5 columns

When a thread needs to execute multiple columns, it does so iteratively. As a result, the execution time largely depends on the thread with the highest number of iterations. In this example, the maximum amount of iterations of the threads for each row is: 5, 3, 2, 2, 1, respectively. If W represents the width of the screen and n the number of threads, the maximum iterations of the threads can be expressed as:

$$x = \lceil W/n \rceil \qquad \forall n \in \mathbb{N} \tag{1}$$

This equation can be inverted to show at which amounts of threads the graph takes a step.

$$step = \lceil W/x \rceil \qquad \forall x \in \mathbb{N} \tag{2}$$

This last equation means that the steps are located at an amount of threads that is divisable by the screen width to get an integer number. This is clearly represented in the example. With this knowledge the graph in Figure 8 can be explained. In the figure above the screenwidth is 1280 px. Because this example has three blocks, W needs to be divided by 3 ($1280/3 = 426$). Using the second equation, the blocksize transitions with steps are:

$$\lceil 426/1 \rceil = 426$$

$$\lceil 426/2 \rceil = 213$$

$$\lceil 426/3 \rceil = 142$$

$$\lceil 426/4 \rceil = 107$$

$$\lceil 426/5 \rceil = 86$$

This goes on for every integer number up to 426, but the steps become less spaced apart and hard to see in a graph. The numbers calculated here are the exact numbers the graph shows. In the graph it is apparant that the gain of fps gets bigger if the graph steps at bigger amounts of threads. However, more threads need to be added to make this jump.

### 5.4 Influence textures

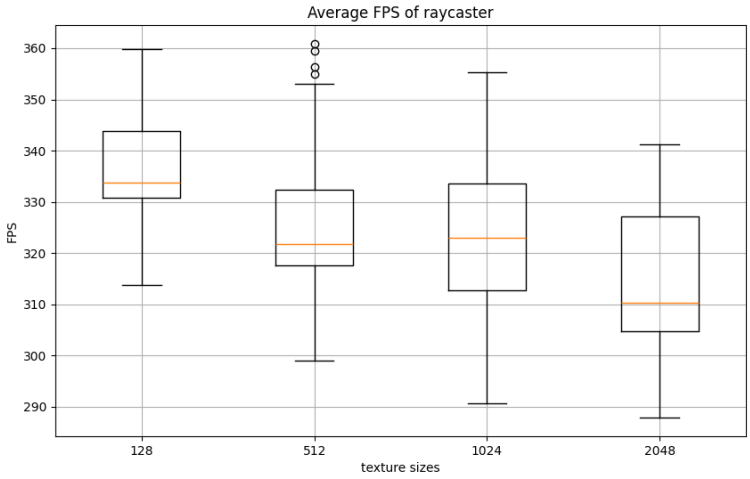



Figure 10: FPS as a function of the blocksize for the different texture sizes (`GPU2`)

The general trend indicates that as the texture sizes increase, the FPS decreases. Additionally, it appears that FPS consistency diminishes. This observation aligns with the process by which textures are rendered, as it involves traversing over the pixels within the texture. With larger textures, more computations are required to determine the necessary pixels, even though the output entails the same number of pixels as smaller textures.

## 6 Conclusion

In conclusion, this project has demonstrated the potential of parallel computing techniques, particularly utilizing CUDA, to enhance the performance of a 2D raycaster. By translating the original Python code into C++ and parallelizing it with CUDA, significant improvements in performance have been achieved. The utilization of parallel processing has allowed for faster rendering of scenes, addressing the issue of fluctuating frame rates observed in the initial implementation.

However, there remain possibilities for further exploration and improvement. One such area is the extension of this work to 3D raycasting. Expanding the capabilities of the raycasting algorithm to handle three-dimensional environments would open up new possibilities. A shortcoming of the 2D raycaster is that the pixels have to be calculated iteratively per column. An extension could be to process each pixel separately instead of each column. In 3D raycasting, a ray is sent through each pixel, and rendering occurs per pixel rather than per column. This not only allows for variations in levels within the game world but also provides more freedom to tune the raycasting process. Currently, we are limited to sending rays per column, but in 3D raycasting, this limitation is lifted, allowing for more flexible rendering techniques. Additionally, exploring rendering techniques such as raytracing could lead to even more realistic and visually appealing graphics.

Furthermore, while CUDA has proven to be a powerful tool for parallelization, there may be alternative parallel computing frameworks or optimizations that could further optimize the raycasting algorithm. Experimenting with different parallelization strategies and optimizations could potentially unlock even greater performance gains.